Modulok: import és export

Részletes útmutató a JavaScript modulrendszeréhez (ESM) Node.js alatt — a kiajánlás és a behozatal minden formájával, a buktatókkal és futtatható példákkal.

1. Miért van szükség modulokra?

Egyetlen fájlba írt program néhány száz sor után áttekinthetetlen. A modul egy **önálló fájl**, amely eldönti, mit mutat meg magából a külvilágnak, és mit tart meg magának. Ami nincs kiajánlva, azt kívülről el sem lehet érni.

Három dolgot nyerünk vele:

- **Áttekinthetőség.** Minden fájlnak egy feladata van.
- **Újrafelhasználás.** Ugyanazt a függvényt több helyről behozhatod.
- **Védelem.** A modul belső segédfüggvényei rejtve maradnak, így nyugodtan átírhatod őket — más kód nem támaszkodhat rájuk.

A modulok nem osztoznak a névtéren: két fájlban lehet ugyanolyan nevű változó, nem zavarják egymást.

2. Két modulrendszer él egymás mellett

A Node.js-ben kétféle modulrendszert találsz. Fontos tudni, melyiket használod, mert az interneten talált példák felváltva mutatják mindkettőt.

	CommonJS (régi)	ES Modules — ezt használjuk
Honnan jön?	`A Node saját megoldása`	`A JavaScript szabványa`
Behozatal	`const x = require("…")`	`import x from "…"`
Kiajánlás	`module.exports = { x }`	`export function x() {}`
Bekapcsolás	`alapértelmezett`	`"type": "module"`
Böngészőben	`nem működik`	`ugyanez a szintaxis működik`
Betöltés	`futás közben, bármikor`	`indulás előtt, elemezve`

Mi végig az ESM-et használjuk, mert ez a szabvány, és ezt ismered a frontendről is. Az utolsó sor a legfontosabb különbség: az ESM-importokat a Node a program indulása **előtt** feldolgozza. Ezért derül ki egy elgépelt modulnév azonnal — és ezért nem lehet a sima importot feltételhez kötni.

3. Bekapcsolás: a package.json

Egyetlen sor kell a projekt `package.json` fájljába:

```
{
  "name": "projektem",
  "type": "module",
  "main": "index.js"
}
```

Ettől kezdve minden `.js` fájl ESM. Ha ez a sor hiányzik, az első `import` utasításnál ezt kapod:

```
SyntaxError: Cannot use import statement outside a module
```

Egyedi fájlokra a kiterjesztéssel is dönthetsz: a `.mjs` mindig ESM, a `.cjs` mindig CommonJS — függetlenül a `package.json`-tól.

4. Kiajánlás — az export

A modul maga dönti el, mit mutat meg. Kétféle kiajánlás van: **nevesített** (tetszőleges számú) és **alapértelmezett** (fájlonként legfeljebb egy).

4.1 Nevesített kiajánlás a deklaráció előtt

```
export function osszead(a, b) {
  return a + b;
}

export const PI = 3.14159;
```

4.2 Nevesített kiajánlás utólag, listával

Ez a forma akkor kényelmes, ha egy helyen akarod látni a modul teljes „felületét”. Előbb megírod a függvényeket, a végén felsorolod, mit adsz ki:

```
function szoroz(a, b) {
  return a * b;
}

function oszt(a, b) {
  if (b === 0) throw new Error("Nullával nem lehet osztani.");
  return a / b;
}

export { szoroz, oszt };
```

4.3 Kiajánlás más néven

A belső név maradhat rövid, a kifelé mutatott lehet beszédesebb:

```
export { kerekit as kerekitesTizedesre };
```

4.4 Alapértelmezett kiajánlás

Azt tesszük ide, ami a modul **fő dolga**. A hívó oldalon kapcsos zárójel nélkül, tetszőleges néven hozható be — a név nincs rögzítve a forrásfájlban.

```
export default function korTerulete(sugar) {
  return PI * sugar * sugar;
}
```

Mikor melyiket? Ha a modul **több, egyenrangú** dolgot ad (segédfüggvények), legyen mind nevesített. Ha **egyetlen** dolgot (egy osztály, egy beállításobjektum, egy szolgáltatás), lehet alapértelmezett. A nevesített a biztonságosabb: az elgépelt nevet a Node azonnal jelzi, az alapértelmezettnél viszont bármit elfogad névnek.

5. Behozatal — az import

5.1 Nevesített behozatal

Kapcsos zárójelben pontosan azokat a neveket sorolod fel, amelyeket a másik fájl kiajánlott. A sorrend nem számít, a név igen. Nem kötelező mindent behozni.

```
import { osszead, kivon, PI } from "./modulok/matek.js";
```

5.2 Alapértelmezett behozatal

```
import korTerulete from "./modulok/matek.js";
import akarmilyenNev from "./modulok/matek.js";   // ugyanaz, más néven
```

5.3 A kettő együtt

Ez a leggyakoribb forma: előbb az alapértelmezett, vessző után a nevesítettek.

```
import korTerulete, { osszead, PI } from "./modulok/matek.js";
```

5.4 A teljes modul egyetlen objektumként

Az `import * as név` a modul **minden** nevesített kiajánlását egy objektumba gyűjti. Ez a „komplett modul behozatala”.

```
import * as matek from "./modulok/matek.js";

matek.osszead(2, 3);
matek.default(2);              // az alapértelmezett „default” néven
Object.keys(matek);            // mit ad a modul?
```

Mikor jó? Ha a hívásnál látni akarod, honnan jön a függvény. A rétegelt felépítésben a vezérlők így hozzák be a szolgáltatásokat: a `szolgaltatas.osszes()` alakból azonnal látszik, melyik réteg dolgozik.

5.5 Átnevezés behozatalkor

Akkor kell, ha két modul ugyanazt a nevet adja, vagy a név túl általános nálunk:

```
import { osszead as osszeadSzamokat, PI as KOR_PI } from "./modulok/matek.js";
```

5.6 Behozatal csak a mellékhatásért

Ez a forma semmit nem hoz be — csak **lefuttatja** a modult. A valódi világban: polyfill betöltése, eseményfigyelő beállítása, bővítmény regisztrálása.

```
import "./modulok/mellekhatas.js";
```

6. A három kötelező szabály az útvonalakban

Ez okozza a legtöbb kezdeti hibát. Node-ban mindhárom kötelező:

Szabály	Rossz	Jó
Saját fájlnál kell a **./** vagy **../**	`from "modulok/matek.js"`	`from "./modulok/matek.js"`
Saját fájlnál kell a **.js** kiterjesztés	`from "./modulok/matek"`	`from "./modulok/matek.js"`
npm-csomagnál **csak a név**	`from "./express"`	`from "express"`

A kiterjesztés azért kötelező, mert a szabvány nem ír elő találgatást. A frontendes építőeszközök (Vite, webpack) kiegészítik helyetted — a Node nem. Ezért másolható egy internetes példa gyakran csak javítással.

7. Élő kötés és gyorsítótár

Két olyan tulajdonság, amit sokan félreértenek — és amire az egész alkalmazásszerkezet épül.

7.1 Az import nem másolat

A behozott név a modul **változójára mutat**. Ha a modul megváltoztatja, a behozó oldalon is az új érték látszik. Ezt hívják élő kötésnek (live binding).

```
import { szamlalo, novel } from "./modulok/allapot.js";

console.log(szamlalo);   // 0
novel();
console.log(szamlalo);   // 1  — magától megváltozott
```

Fordítva viszont nem megy: a behozott nevet **mi nem írhatjuk át**. A `szamlalo = 10` sor `TypeError: Assignment to constant variable` hibát ad. Csak a modul maga módosíthatja, a saját függvényein keresztül.

7.2 A modul egyszer fut le

Akárhány fájl hozza be ugyanazt a modult, a Node **egyszer** tölti be és futtatja le; minden további behozatal ugyanazt a példányt kapja.

Erre épül a gyakorlat. Az adatbázis-kapcsolat, a beállításobjektum és a naplózó egyszer jön létre, és minden fájl ugyanazt használja. Nem kell körbeadogatni őket.

7.3 Az importok felcsúsznak

A Node az importokat a fájl minden más utasítása **előtt** feldolgozza. Ezért akkor is a behozott modul kimenete jelenik meg először, ha az `import` a fájl közepén áll.

8. Újraexportálás — több modul egy belépési ponton

A gyűjtőfájl nem ír saját kódot: összegyűjti más modulok kiajánlásait. A használó így egyetlen helyről hoz be mindent, és nem kell tudnia, melyik függvény melyik fájlban van. A könyvtárak `index.js` fájlja pontosan ezt csinálja.

```
// modulok/gyujto.js
export * from "./szoveg.js";                          // minden nevesített
export { osszead, PI as KOR_PI } from "./matek.js";   // válogatás + átnevezés
export { default as korTerulete } from "./matek.js";  // az alapértelmezett
export { default as naplo } from "./naplo.js";

// a használó oldalon
import { osszead, tisztit, korTerulete, naplo } from "./modulok/gyujto.js";
```

Az `export *` az alapértelmezett kiajánlást NEM viszi tovább. Ez logikus: több forrásnak több alapértelmezettje lenne, és csak egy lehet. Ezért kell kézzel, nevesítetté alakítva továbbadni — lásd a harmadik és negyedik sort fent.

9. Dinamikus import — behozatal futás közben

A sima `import` a fájl tetején áll, és nem lehet feltételhez kötni. Az `import()` viszont függvényként viselkedik: bárhol meghívható, `Promise`-t ad vissza, és a modul neve lehet változó.

```
// feltételes betöltés
if (kell) {
  const matek = await import("./modulok/matek.js");
  matek.osszead(2, 3);
}

// a név lehet változó
const modul = await import(`./modulok/${melyiket}.js`);

// és a hiba ELKAPHATÓ
try {
  await import("./nincs-ilyen.js");
} catch (hiba) {
  console.log(hiba.code);        // ERR_MODULE_NOT_FOUND
}
```

Az `import()` **mindig névtérobjektumot** ad: az alapértelmezett kiajánlás a `.default` mezőben van, akkor is, ha a modulnak csak az van.

Mikor használjuk? Ritkán kellő, nagy modulnál; bővítmények név szerinti betöltésénél; feltételtől függő betöltésnél. Alapesetben maradj a sima importnál — az gyorsabb és átláthatóbb.

10. JSON behozatala

JSON-fájlt is be lehet hozni modulként, de kötelező megadni a típust. Ez biztonsági intézkedés: enélkül egy kiszolgáló becsaphatna úgy, hogy JSON helyett futtatható kódot küld.

```
import beallitasok from "./modulok/beallitasok.json" with { type: "json" };

console.log(beallitasok.port);   // 3000 – szám, nem szöveg

// dinamikusan:
const m = await import("./beallitasok.json", { with: { type: "json" } });
```

Három dolgot érdemes tudni:

- a `with { type: "json" }` rész **kötelező**,
- a JSON mindig az **alapértelmezett** kiajánlásba kerül — mezőit nevesítve nem lehet behozni,
- a fájl a program indulásakor **egyszer** töltődik be; ha a tartalom futás közben változhat, ne importáld, hanem olvasd be az `fs` modullal.

11. import.meta — hol vagyok?

ESM-ben nincs `__dirname` és `__filename`. Helyettük az `import.meta` objektum áll rendelkezésre.

Mező	Mit ad?
`import.meta.dirname`	a modul mappája (a régi __dirname) — Node 21.2 óta
`import.meta.filename`	a fájl teljes útvonala (a régi __filename)
`import.meta.url`	a modul címe URL-ként — a legrégebbi, mindenhol működő forma
`import.meta.resolve(x)`	hová oldódna fel egy import — betöltés nélkül

Ez nem ugyanaz, mint a `process.cwd()`. Az azt mondja meg, **honnan indítottad** a programot — és megváltozik, ha máshonnan indítod. Az `import.meta.dirname` a fájl helye, és soha nem változik. Fájlt ezért

mindig ebből építve nyiss meg:

```
path.join(import.meta.dirname, "adatok.json")
```

12. Körkörös import

Körkörös a hivatkozás, ha A behozza B-t, B pedig A-t. Az ESM ezt nem tiltja, de a viselkedés attól függ, **mit** használsz a körben.

Lemérve, Node 24-en:

```
// a.js
import { bNev } from "./b.js";
export const aNev = "A";
console.log(bNev);

// b.js
import { aNev } from "./a.js";
export const bNev = "B";
console.log(aNev);        // ReferenceError: Cannot access 'aNev' before initialization
```

Az `a.js`-t indítva a kimenet első sora a `b.js`-ből jön (az import felcsúszik), és a `const` még nincs inicializálva, amikor a `b.js` használni akarja.

Függvényekkel viszont működik — azok felcsúsznak a modul elejére:

```
// c.js
import { bKoszon } from "./d.js";
export function aKoszon() { return "A köszön"; }
console.log(bKoszon());   // működik

// d.js
import { aKoszon } from "./c.js";
export function bKoszon() { return "B köszön (és tudja: " + aKoszon() + ")"; }
```

Mit tegyél? A körkörös hivatkozás majdnem mindig tervezési hiba jele. A szokásos megoldás: a közös részt kiemelni egy harmadik modulba, amelyet mindkettő behoz. A rétegelt felépítés (útvonal → vezérlő → szolgáltatás) eleve kizárja a kört, mert a függés egyirányú.

13. A hat leggyakoribb hiba

Az alábbi üzenetek **valódi futtatásból** származnak (Node 24.20.0). A modulhibák többsége a program indulása előtt csap le, ezért `try/catch`-csel nem lehet őket elkapni.

Mit rontottál el?	Mit mond a Node?	Javítás
Lemaradt a **.js**	`ERR_MODULE_NOT_FOUND` `Cannot find module …`	Írd ki a kiterjesztést
Lemaradt a **./**	`ERR_MODULE_NOT_FOUND` `Cannot find package …`	Saját fájlnál kell a ./ vagy ../
Nem létező nevet hozol be	`SyntaxError: … does not provide an export named …`	Nézd meg: import * as m, majd Object.keys(m)
require() ESM-ben	`ReferenceError: require is not defined in ES module scope`	import-ot használj, vagy import()-ot

Mit rontottál el?	Mit mond a Node?	Javítás
Átírod a behozott nevet	`TypeError: Assignment to constant variable.`	Kérd meg a modult egy függvénnyel
Hiányzik a "type": "module"	`SyntaxError: Cannot use import statement outside a module`	Vedd fel a package.json-be

A második sorban a különbség árulkodó: ha lemarad a `./`, a Node nem fájlt, hanem **csomagot** keres — ezért más a hibaüzenet, mint az első sorban.

14. Összefoglaló táblázat

Amit szeretnél	Így írod
Néhány függvényt behozni	`import { a, b } from "./m.js"`
Az alapértelmezettet behozni	`import barmi from "./m.js"`
Mindkettőt egyszerre	`import barmi, { a } from "./m.js"`
A **teljes modult** objektumként	`import * as m from "./m.js"`
Más néven behozni	`import { a as b } from "./m.js"`
Csak lefuttatni a modult	`import "./m.js"`
Futás közben betölteni	`const m = await import("./m.js")`
JSON-t behozni	`import j from "./a.json" with { type: "json" }`
Függvényt kiajánlani	`export function a() {}`
Utólag, listával kiajánlani	`export { a, b }`
Más néven kiajánlani	`export { a as b }`
Alapértelmezettet kiajánlani	`export default …`
Továbbadni mindent	`export * from "./m.js"`
Továbbadni az alapértelmezettet	`export { default as m } from "./m.js"`

15. A mellékelt példafájlok

A leírás mellett egy futtatható példacsomag is tartozik ehhez az anyaghoz. Minden fájl önállóan indítható, és a végén „PRÓBÁLD KI!” szakasz mondja meg, mit érdemes elrontani, hogy lásd a következményt.

```
cd pelda
node 01-nevesitett-import.js
```

Fájl	Mit mutat meg?
`01-nevesitett-import.js`	Nevesített behozatal, válogatás, hiba a modulból
`02-default-import.js`	Alapértelmezett behozatal, tetszőleges néven
`03-nevter-import.js`	A **teljes modul** objektumként; a modulobjektum nem írható

Fájl	Mit mutat meg?
`04-atnevezes.js`	Átnevezés mindkét oldalon (as)
`05-ujraexport.js`	Gyűjtőmodul; mit nem visz tovább az export *
`06-mellekhatas.js`	Behozatal csak a mellékhatásért; az importok felcsúszása
`07-elo-kotes.js`	Élő kötés és a modulok gyorsítótára
`08-dinamikus-import.js`	import() feltétellel, változó névvel, hibakezeléssel
`09-json-import.js`	JSON behozatala a with jelöléssel
`10-import-meta.js`	dirname, filename, url, resolve — és a cwd különbsége
`11-gyakori-hibak.js`	A hat hiba **valódi** üzenete, külön folyamatokban futtatva

A `modulok/` mappában vannak a behozott modulok (`matek.js`, `szoveg.js`, `naplo.js`, `allapot.js`, `mellekhatas.js`, `gyujto.js`, `beallitasok.json`), a `hibak/` mappában pedig a szándékosan hibás fájlok — ezeket a 11. példa futtatja le.